

TABLAS DE DISPERSIÓN

Hash Tables — Direccionamiento Abierto y Aplicaciones

Estructura de Datos — Semana 11

Grupo 2

Integrante 1 · Integrante 2 · Integrante 3 · **Frank Abel Chambi Condori**

Universidad Continental | Ingeniería de Sistemas e Informática | 2026

¿Qué aprendió el Grupo 1?

 Integrante 1

Resumen rápido (2 min):

- ▶ Hash table = arreglo + función hash $\rightarrow O(1)$
- ▶ Factor de carga $\alpha = n/m \rightarrow$ mantener < 0.7
- ▶ Chaining: lista enlazada en cada bucket

Hoy el Grupo 2 cubre:

- ▶ Otra forma de manejar colisiones $\rightarrow$ **Open Addressing**
- ▶ Tres estrategias de sondeo
- ▶ Aplicación real: contador de palabras
- ▶ ¿Qué usa Python, Java, Redis?

Open Addressing — La Idea

Integrante 1

Sin listas enlazadas. Todo dentro del arreglo.

Cuando hay colisión → busco el **siguiente bucket disponible** (sondeo).

```
 $\alpha$  debe ser SIEMPRE  $< 1$   
(más elementos que cubetas = imposible)
```

Tres estrategias de sondeo:

1. Sondeo Lineal
2. Sondeo Cuadrático
3. Doble Hashing ← el mejor

Sondeo Lineal

Integrante 1

```
h(k, i) = (h'(k) + i) mod m,    con i = 0, 1, 2, ...
```

Si el bucket está ocupado → prueba el siguiente consecutivo.

```
[0]    [1]    [2]Ana  [3]Luis  [4]Pedro  [5]    [6]  
          ←—— clustering primario ———→
```

Problema: Clustering Primario → cadenas largas → rendimiento cae.

Analogía: el tráfico en una autopista — si un carril se llena, los autos se apilan.

Sondeo Cuadrático y Doble Hashing

 Integrante 2

Sondeo Cuadrático:

```
h(k, i) = (h'(k) + i2) mod m  
Pasos: +1, +4, +9, +16, ...
```

Reduce clustering primario, pero genera clustering secundario.

Doble Hashing (el mejor):

```
h(k, i) = (h1(k) + i × h2(k)) mod m  
h2(k) = 1 + (hashCode(k) % (m-1)) ← nunca 0
```

El paso depende de la clave → distribución casi perfecta.

Lazy Deletion — Eliminación Inteligente

Integrante 2

El problema: si borramos un elemento, dejamos un hueco que interrumpe búsquedas.

```
Buscar "Carlos":  
[2] OCUPADO ("Ana")    → no es Carlos, sigo  
[3] VACÍO              → STOP, no existe (¡pero Carlos sí está en [4]!)
```

La solución: marcar como ELIMINADO, no limpiar.

```
enum Estado { VACIO, OCUPADO, ELIMINADO };  
// Las búsquedas continúan pasando por ELIMINADO  
// Las inserciones pueden reutilizar ELIMINADO
```

Estructura en C++ (sin STL)

Integrante 3

```
enum Estado { VACIO, OCUPADO, ELIMINADO };

struct Entrada {
    string clave;
    string valor;
    Estado estado;
};

struct TablaHash {
    int tamano;
    int numElementos;
    Entrada* tabla;

    int h1(string clave);    // hash primario - djb2
    int h2(string clave);    // hash secundario (≠ 0)
    int sondear(string clave, int i, int metodo);
    int insertar(string clave, string valor, int metodo);
    string buscar(string clave, int metodo, int& pasos);
    bool eliminar(string clave, int metodo);
};
```

Demo — Inserción con Doble Hashing

Integrante 3

```
Sistema - programa_principal.exe (Doble Hashing)

C:\Grupo2_Semana11_HashTables\codigo> programa_principal.exe

Seleccione metodo de sondeo:
 1 = Lineal, 2 = Cuadratico, 3 = Doble Hashing
Opcion: 3

===== AGENDA DE MATERIAS (Open Addressing) =====
[1] Insertar materia
[2] Buscar materia
[3] Eliminar materia
[4] Mostrar tabla completa
[5] Ejecutar comparativa Lineal vs Doble Hashing
[6] Cambiar metodo de sondeo
[0] Salir
Opcion: 1
Codigo: INF101
Nombre: Algoritmica I
  >> Insertado en 1 paso(s).

Opcion: 1
Codigo: INF102
Nombre: Algoritmica II
  >> Insertado en 1 paso(s).

Opcion: 1
Codigo: INF103
Nombre: Estr. Datos
  >> Insertado en 1 paso(s).

Opcion: 4

+-----+-----+-----+-----+
| Pos | Clave | Valor          | Estado |
+-----+-----+-----+-----+
| 0   |      |                | VACIO  |
| 1   | INF101 | Algoritmica I  | OCUPADO|
| 2   | INF102 | Algoritmica II | OCUPADO|
| 3   | INF103 | Estr. Datos    | OCUPADO|
| 4   |      |                | VACIO  |
| 5   |      |                | VACIO  |
| 6   |      |                | VACIO  |
| 7   |      |                | VACIO  |
| 8   |      |                | VACIO  |
| 9   |      |                | VACIO  |
| 10  |      |                | VACIO  |
+-----+-----+-----+-----+
Elementos: 3 | Factor de carga: 0.27

Opcion: 0
```


Comparativa — Mismo dato, dos métodos

 Integrante 3

Clave	Sondeo Lineal	Doble Hashing
INF101	1 paso	1 paso
INF102	1 paso	1 paso
INF103	1 paso	1 paso
MAT201	4 pasos	2 pasos
MAT202	4 pasos	2 pasos
FIS301	6 pasos	2 pasos
TOTAL	17 pasos	9 pasos
Promedio	2.83	1.50

Doble hashing: 47 % menos pasos que sondeo lineal.

Evidencia — Comparativa en consola

 Integrante 3

```
Sistema - programa_principal.exe (Comparativa)

C:\Grupo2_Semana11_HashTables\codigo> programa_principal.exe

Seleccione metodo de sondeo:
 1 = Lineal, 2 = Cuadratico, 3 = Doble Hashing
Opcion: 1

===== AGENDA DE MATERIAS (Open Addressing) =====
[1] Insertar materia
[2] Buscar materia
[3] Eliminar materia
[4] Mostrar tabla completa
[5] Ejecutar comparativa Lineal vs Doble Hashing
[6] Cambiar metodo de sondeo
[0] Salir
Opcion: 5

=====
COMPARATIVA: Lineal vs Doble Hashing
=====
```

Clave	Lineal	Doble Hash
INF101	1	1
INF102	1	1
INF103	1	1
MAT201	4	2
MAT202	4	2
FIS301	6	2
TOTAL	17	9
Promedio	2.83	1.50

```
>> Doble Hashing realiza 47% menos pasos que Sondeo Lineal.

Opcion: 0
```

Demo — Lazy Deletion en vivo

Integrante 3

```
Sistema - programa_principal.exe (Lazy Deletion)

C:\Grupo2_Semana11_HashTables\codigo> programa_principal.exe

Seleccione metodo de sondeo:
Opcion: 1

Opcion: 1 (inserto INF101) >> Insertado en 1 paso(s).
Opcion: 1 (inserto INF102) >> Insertado en 1 paso(s).
Opcion: 1 (inserto INF103) >> Insertado en 1 paso(s).
Opcion: 1 (inserto MAT201) >> Insertado en 4 paso(s).

Opcion: 4
+-----+-----+-----+-----+
| Pos | Clave | Valor | Estado |
+-----+-----+-----+-----+
| 1 | INF101 | Algoritmica I | OCUPADO |
| 2 | INF102 | Algoritmica II | OCUPADO |
| 3 | INF103 | Estr. Datos | OCUPADO |
| 4 | MAT201 | Calculo I | OCUPADO |
+-----+-----+-----+-----+
Elementos: 4 | Factor de carga: 0.36

Opcion: 3
Codigo a eliminar: INF102
>> Eliminada (marcada como ELIMINADO).

Opcion: 4
+-----+-----+-----+-----+
| Pos | Clave | Valor | Estado |
+-----+-----+-----+-----+
| 1 | INF101 | Algoritmica I | OCUPADO |
| 2 | ---- | | ELIMINADO | <-- marcador
| 3 | INF103 | Estr. Datos | OCUPADO |
| 4 | MAT201 | Calculo I | OCUPADO |
+-----+-----+-----+-----+
Elementos: 3 | Factor de carga: 0.27

Opcion: 2
Codigo a buscar: INF103
>> Encontrada: Estr. Datos (1 pasos)

>> La busqueda CRUZA la cubeta ELIMINADO y encuentra INF103 OK.

Opcion: 0
```

Complejidad — Open Addressing

 Integrante 3

Operación	Promedio ($\alpha < 1$)	Peor caso
Inserción	$O(1/(1-\alpha))$	$O(n)$
Búsqueda exitosa	$O((1/\alpha) \cdot \ln(1/(1-\alpha)))$	$O(n)$
Búsqueda fallida	$O(1/(1-\alpha))$	$O(n)$
Eliminación (lazy)	$O(1/(1-\alpha))$	$O(n)$

Interpretación práctica:

- ▶ $\alpha = 0.5 \rightarrow \approx 2$ pasos por operación
- ▶ $\alpha = 0.7 \rightarrow \approx 3.3$ pasos
- ▶ $\alpha = 0.9 \rightarrow \approx 10$ pasos ⚠

Chaining vs Open Addressing

 Integrante 3

Aspecto	Chaining	Open Addressing
Memoria	Extra (punteros)	Solo arreglo
Factor de carga	Puede ser > 1	Debe ser < 1
Caché CPU	Pobre	Bueno
Implementación	Sencilla	Compleja (lazy del.)
Hash mediocre	Tolerante	Sensible

¿Cuándo cada uno? Chaining → cantidad muy dinámica. Open Addressing → rendimiento crítico y memoria limitada.

Aplicación — Contador de Palabras

 Frank Abel Chambi Condori

```
Sistema - programa_aplicacion.exe (Contador de palabras)

C:\Grupo2_Semana11_HashTables\codigo> programa_aplicacion.exe

===== CONTADOR DE PALABRAS (Hash Table) =====
Ingrese un texto (terminar con ENTER):
el amor es amor y el amor no miente nunca

>> Total de palabras procesadas: 10

=== TOP 3 palabras mas frecuentes ===
1. "amor" -> 3 veces
2. "el" -> 2 veces
3. "es" -> 1 veces

C:\Grupo2_Semana11_HashTables\codigo>
```

¿Dónde se usa esto? Análisis de sentimientos · Motores de búsqueda · Detección de spam

¿Qué usa la industria?

 Frank Abel Chambi Condori

Sistema	Técnica	Por qué
Python dict	Open Addressing (cuadrático perturbado)	Rápido, RAM compacta
Java HashMap	Chaining (árbol al saturar)	Seguro en cualquier α
C++ unordered_map	Chaining	Estándar
Redis	Hash con rehashing incremental	Caché distribuida
DNS	Hash distribuida	Millones de queries/seg
Git	Hash SHA-1	Identidad de commits

Vibe Coding — Lo que usamos

 Frank Abel Chambi Condori

Herramienta: Claude (Anthropic)

Prompt:

"Escribe en C++ (sin STL, compatible con Dev C++) una tabla hash con direccionamiento abierto: 3 métodos de sondeo, lazy deletion, menú interactivo, comparativa que cuente pasos."

Errores que detectamos:

- ▶ `h2(k)` retornaba 0 → corregimos a `1 + (hashCode % (m-1))`
- ▶ La búsqueda se cortaba en ELIMINADO → corregimos: solo VACIO corta
- ▶ Caracteres acentuados en consola Windows → reemplazamos por ASCII

Reflexión: La IA acelera el esqueleto, pero solo si entiendes la teoría detectas los bugs sutiles.

Conclusiones

 Frank Abel Chambi Condori

Hash Tables en una frase:

Arreglo + función hash + manejo de colisiones = $O(1)$ para todo.

Recuerda:

- ▶ Tabla de tamaño **primo**.
- ▶ $\alpha < 0.7$ o redimensiona.
- ▶ Chaining para cargas altas; Open Addressing para velocidad y caché.
- ▶ **Doble hashing** > **Cuadrático** > **Lineal** en distribución.
- ▶ Lazy Deletion es OBLIGATORIA en Open Addressing.

En la industria: Python, Java, Redis, DNS, Git — todos usan hash tables.

| Gracias

 Frank Abel Chambi Condori

¿Preguntas?

Grupo 2 — Integrante 1 · Integrante 2 · Integrante 3 · Frank Abel Chambi Condori

Código disponible en: `Grupo2_Semana11_HashTables/codigo/`

Herramienta de IA usada: Claude (Anthropic)